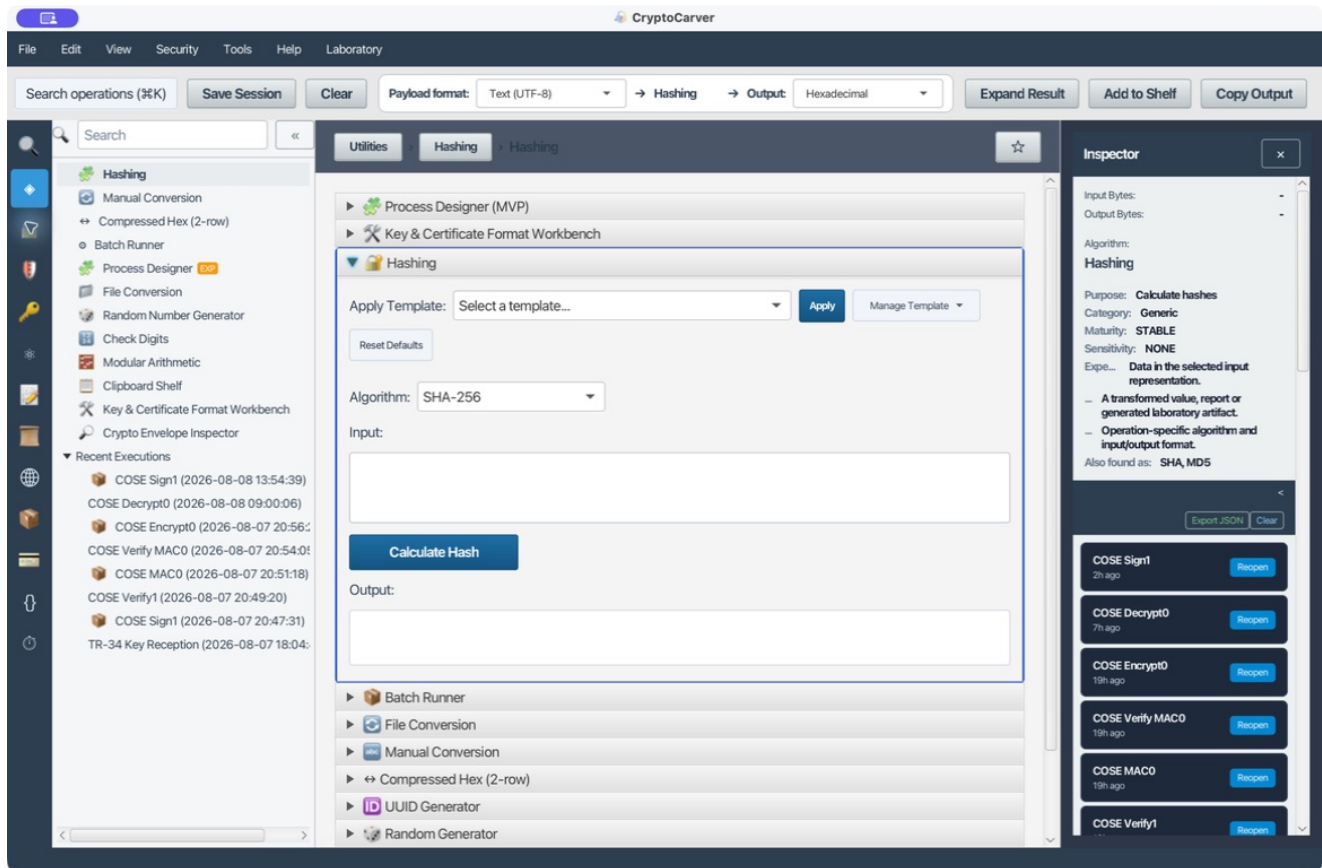


Operaciones Genéricas, Formatos y Flujos Reproducibles



Las operaciones genéricas son el banco de trabajo de CryptoCarver: preparan bytes, validan representaciones, conservan evidencias y permiten inspeccionar artefactos antes de llevarlos a una operación criptográfica. Este tutorial está organizado por intención: qué quieres hacer, con qué entrada verificable y qué resultado debe aparecer.

Trabaja con datos de laboratorio. El historial y Clipboard Shelf pueden conservar valores. No pegues claves privadas, PIN, credenciales ni información de producción.

Mapa de decisión: Qué quiero hacer

Quiero conseguir esto	Herramienta	Comprobación final
Calcular una huella reproducible	Hashing	Algoritmo, bytes y digest completos
Cambiar de representación	Manual Conversion	Round-trip y longitud de bytes
Intercalar un hexadecimal de dos filas	Compressed Hex (2-row)	Filas y valor intercalado
Procesar varias entradas	Batch Runner	Recuento y error por fila
Diseñar un flujo seguro	Process Designer	Dry Run y contrato de cada nodo
Tratar un archivo sin cargarlo entero	File Conversion	Tamaño y hash streaming
Crear un identificador público	UUID Generator	UUID v4 con formato correcto
Obtener bytes de entropía	Random Number Generator	Longitud solicitada y finalidad
Detectar errores de captura	Check Digits	Dígito y número completo
Verificar un cálculo de protocolo	Modular Arithmetic	Operación, operandos y módulo
Reutilizar evidencia de sesión	Clipboard Shelf	Clasificación, origen y formato
Entender un PEM, DER o JWK	Key & Certificate Format Workbench	Tipo, algoritmo y fingerprint
Leer metadatos de un contenedor	Crypto Envelope Inspector	Algoritmo, receptor, KCV y tamaño
Convertir un timestamp Unix a fecha legible	Epoch Converter	Instante UTC en formato ISO-8601
Formatear o validar la sintaxis de un JSON	JSON Formatter	JSON indentado o mensaje de error de parseo

Antes de ejecutar: formato y bytes

El selector global Payload format aplica al contenido ordinario; una clave, IV, nonce, firma o certificado puede tener su propio formato. Un resultado inesperado casi siempre implica una diferencia de bytes: espacio final, salto de línea, 0x, hexadecimal impar, UTF-8 frente a otra codificación o Base64URL frente a Base64.

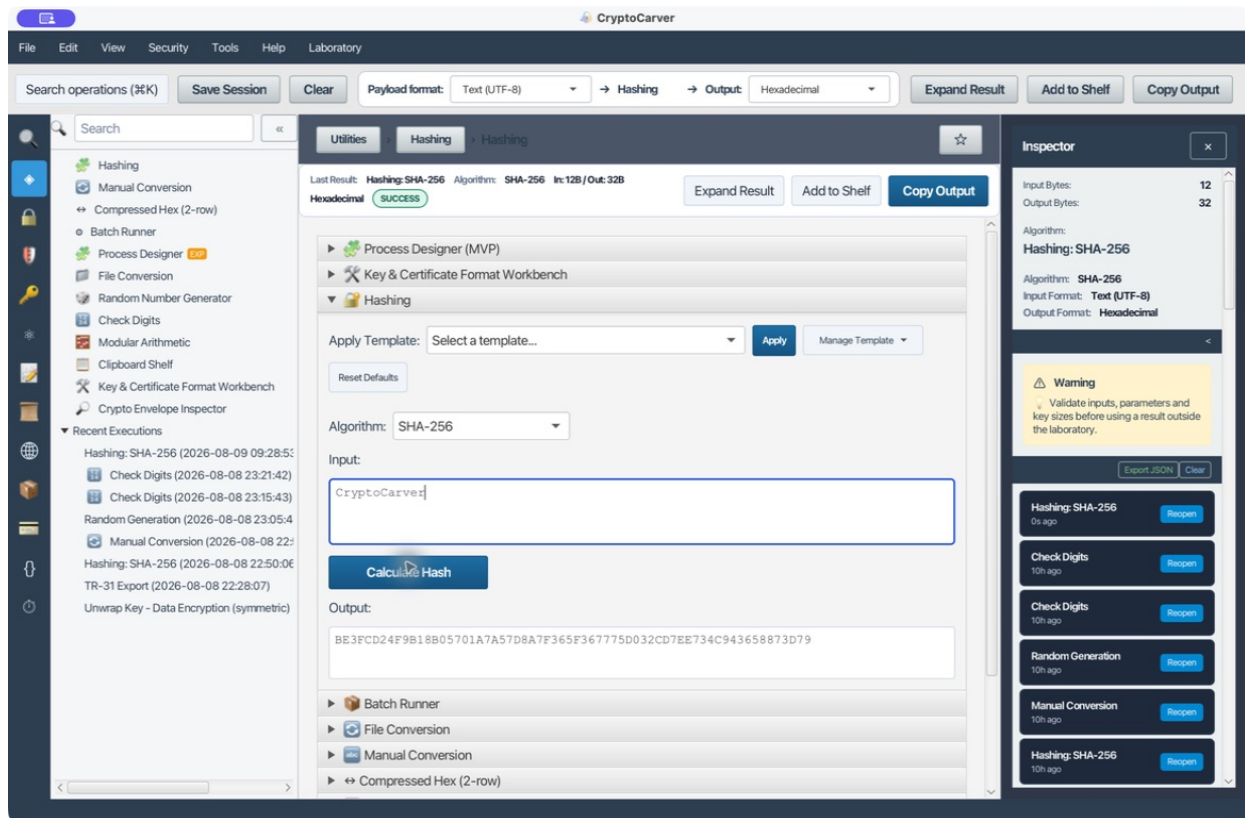
Representación	La persona escribe	La operación recibe
Text (UTF-8)	Caracteres	Bytes UTF-8
Hexadecimal	Dos caracteres por byte	Los bytes indicados
Base64	Texto codificado	Bytes recuperados
Base64URL	Variante apta para URL	Bytes recuperados sin ambigüedad

Caso 1: Quiero demostrar que un texto no ha cambiado

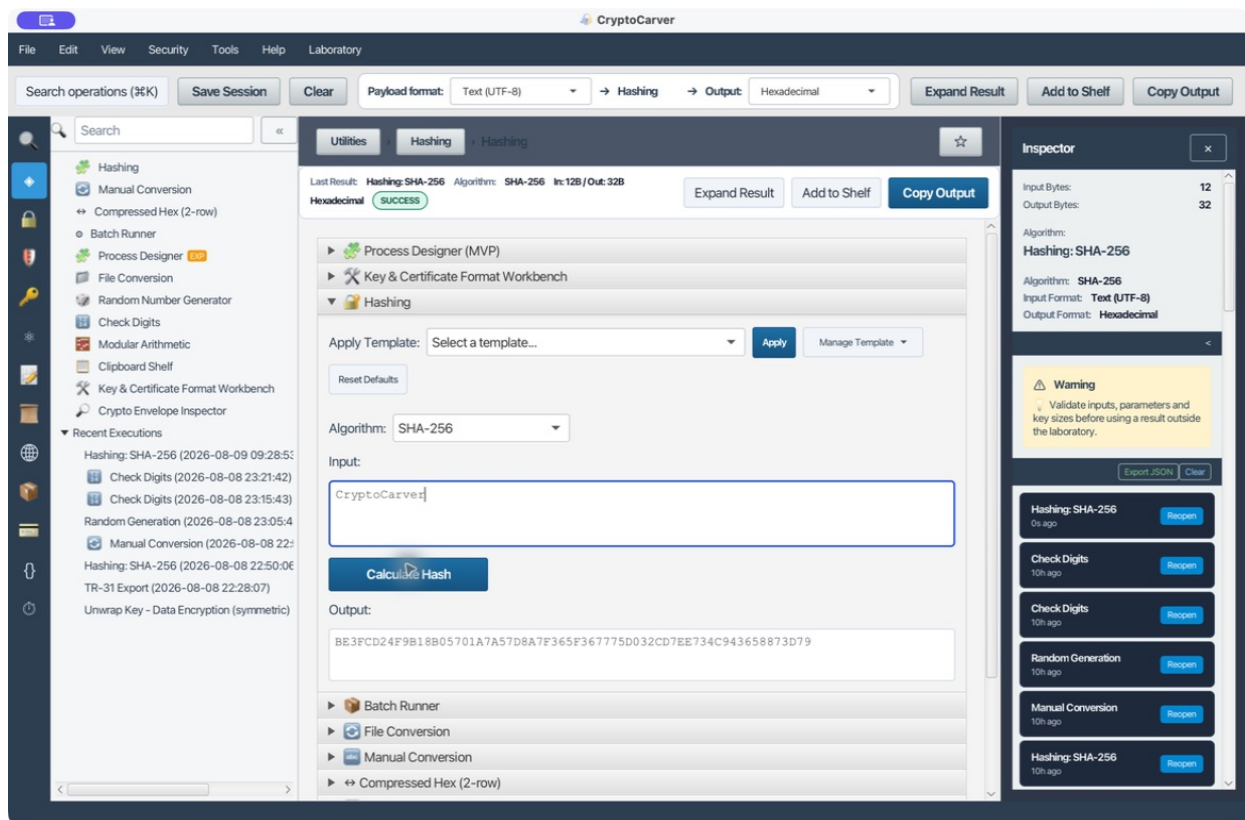
Datos de laboratorio

Parámetro	Valor
Operación	Generic > Hashing
Payload format	Text (UTF-8)
Algoritmo	SHA-256
Entrada	CryptoCarver
Salida	BE3FCD24F9B18B05701A7A57D8A7F365F367775D032CD7E E734C943658873D79

- 1 Abre Hashing, deja SHA-256 y escribe CryptoCarver, sin comillas ni salto final.
- 2 Pulsa Calculate Hash.
- 3 Compara los 64 caracteres del resultado, no solo su prefijo.



Hash SHA-256 reproducible de CryptoCarver



Evidencia del caso 1: hash SHA-256 reproducible de CryptoCarver

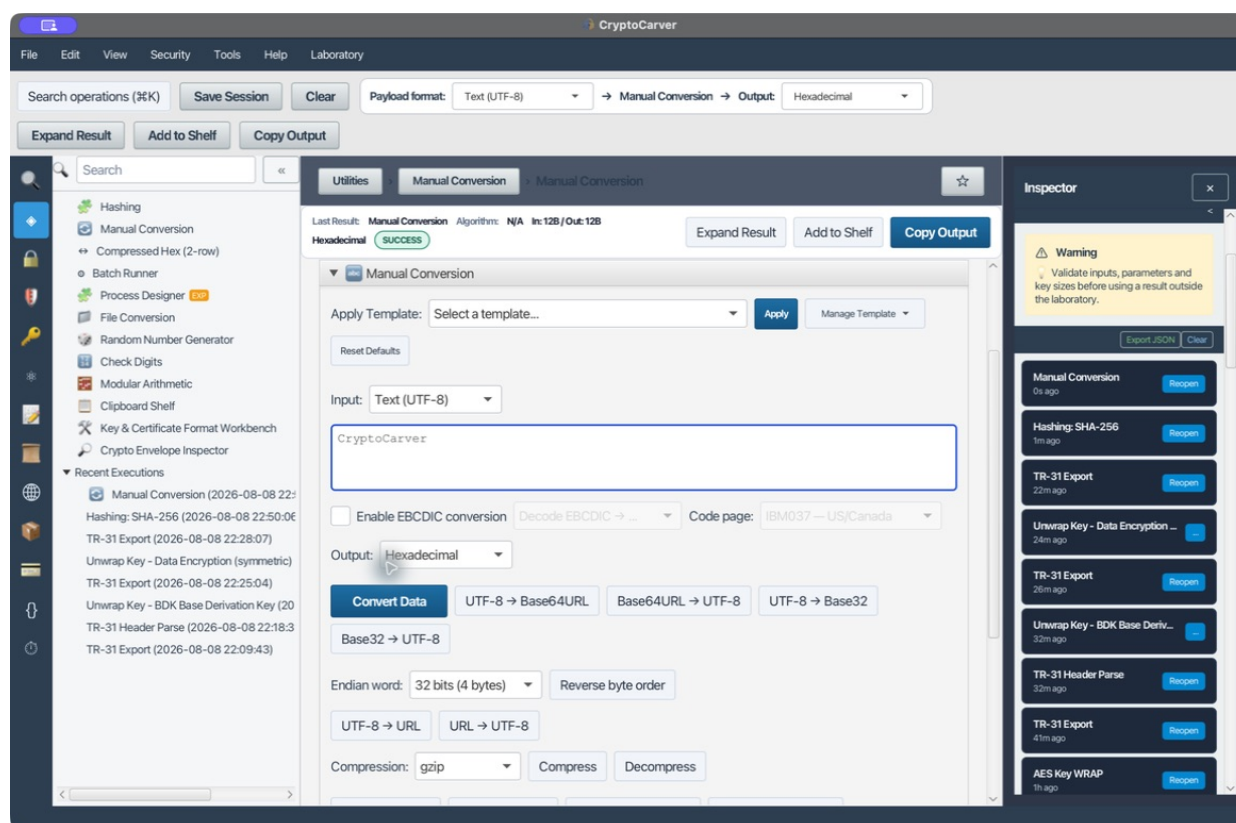
Como prueba negativa, añade un espacio final: el hash debe cambiar por completo. Un hash detecta diferencias cuando el digest de referencia es confiable; no cifra ni autentica por sí solo a un emisor y no sustituye una KDF de contraseñas.

Caso 2: Quiero cambiar la representación sin cambiar los bytes

Datos de laboratorio

Representación	Valor
Texto UTF-8	CryptoCarver
Hexadecimal	43727970746F436172766572
Base64	Q3J5cHRvQ2Fyd mV y
Base64URL	Q3J5cHRvQ2Fyd mV y

- 1 Abre Generic > Manual Conversion.
- 2 Elige Text (UTF-8) como entrada y Hexadecimal como salida.
- 3 Introduce CryptoCarver y pulsa Convert Data.
- 4 Invierte el recorrido: hexadecimal a UTF-8. Debe recuperarse exactamente el texto original.



Conversión de UTF-8 a hexadecimal realizada en CryptoCarver

La conversión no cifra: hexadecimal y Base64 son formas de representar bytes. La misma pantalla permite UTF-8 ↔ URL, Base64URL, EBCDIC, orden de bytes por palabra, gzip y formatos numéricos heredados como BCD o COMP-3. Declara siempre página de códigos, tamaño de palabra, signo y escala cuando intervengan sistemas legados.

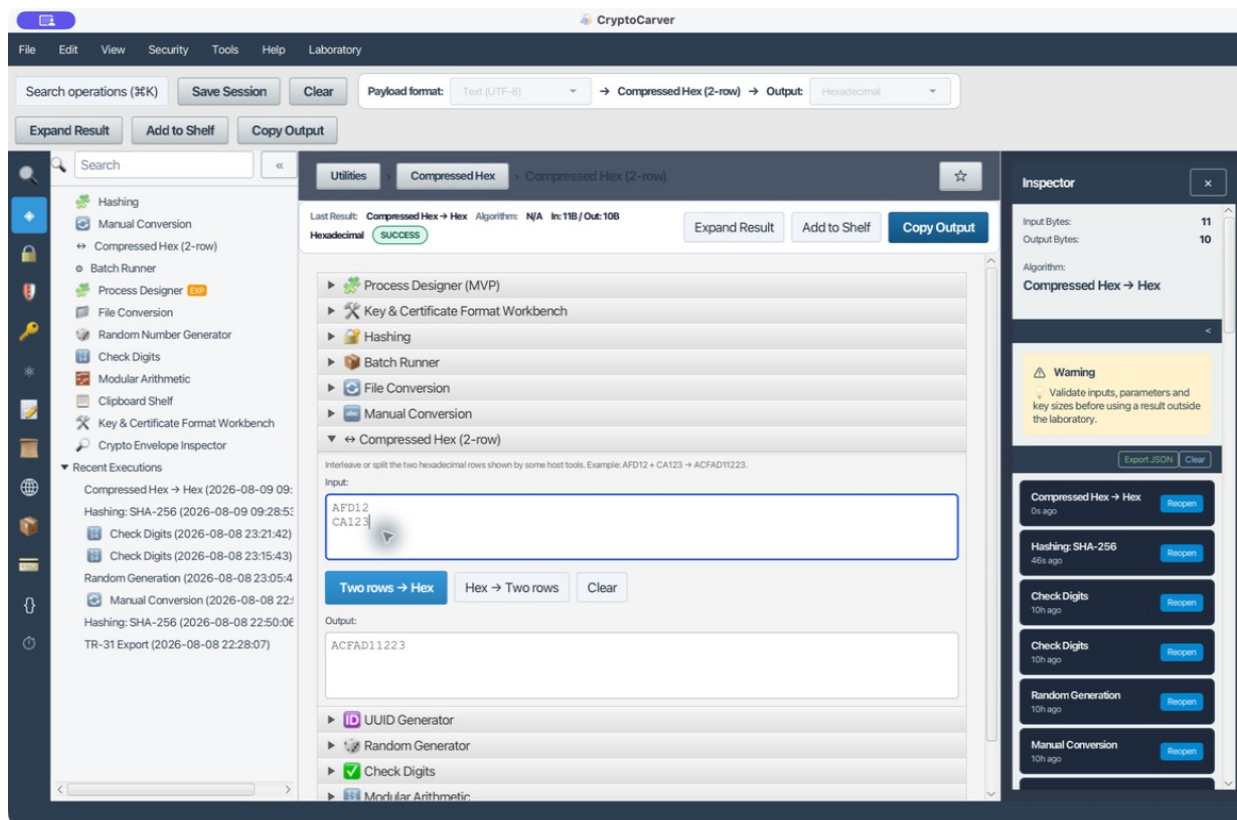
Pruebas negativas útiles: 437 debe fallar por longitud hexadecimal impar; é en UTF-8 debe producir C3A9; y un Base64URL que incluya - o _ no debe procesarse como Base64 ordinario.

Caso 3: Quiero intercalar un hexadecimal de dos filas

Algunos equipos host muestran los nibbles de un campo en dos filas. Compressed Hex (2-row) permite convertir esa forma visual sin transcribir nibble a nibble.

Campo	Valor de laboratorio
Fila 1	AFD12
Fila 2	CA123
Operación	Two rows → Hex
Salida esperada	ACFAD11223

- 1 Abre Generic > Compressed Hex (2-row).
- 2 Pega AFD12 en la primera línea y CA123 en la segunda.
- 3 Pulsa Two rows → Hex y conserva las filas junto al valor resultado para poder revisar la conversión.



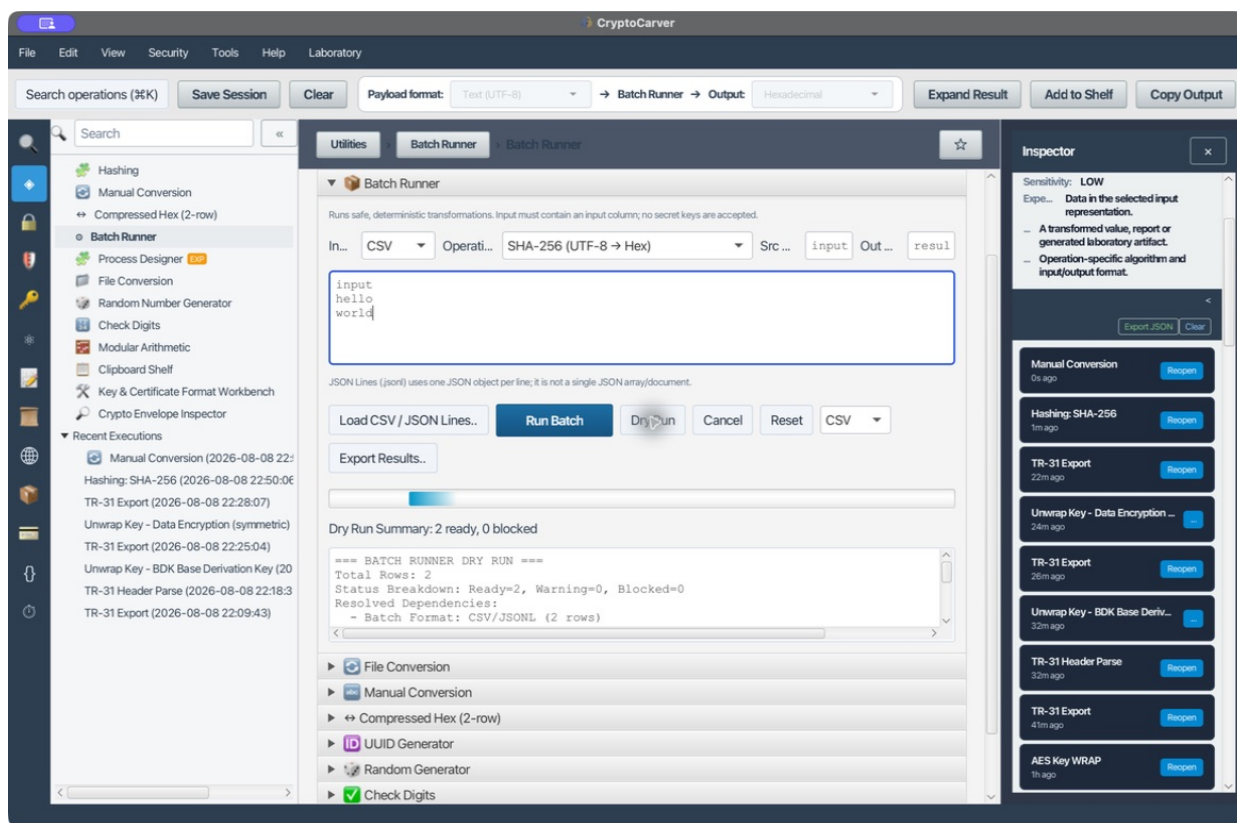
Intercalado de dos filas hexadecimales y resultado ACFAD11223

La operación inversa, Hex → Two rows, es una comprobación excelente: al aplicarla sobre ACFAD11223 deben reaparecer las mismas dos filas. No introduzcas separadores, 0x ni caracteres no hexadecimales.

Caso 4: Quiero preparar un lote de mensajes para verificar su integridad

El objetivo no es “procesar mucho”, sino no perder el vínculo entre cada entrada y su salida. En Batch Runner, el laboratorio de la captura usa dos filas de texto y un Dry Run de SHA-256 (UTF-8 → Hex).

- 1 Abre Generic > Batch Runner y pega o carga un CSV/JSONL con una entrada por registro.
- 2 Declara el formato común de la columna antes de elegir el algoritmo.
- 3 Ejecuta Dry Run: debe indicar cuántas filas están listas y cuántas están bloqueadas, sin efectuar operaciones criptográficas ni crear archivos.
- 4 Con datos aprobados, ejecuta y exporta JSONL; conserva identificador, orden, estado y digest por fila.



Dry Run de un lote SHA-256 con dos filas listas

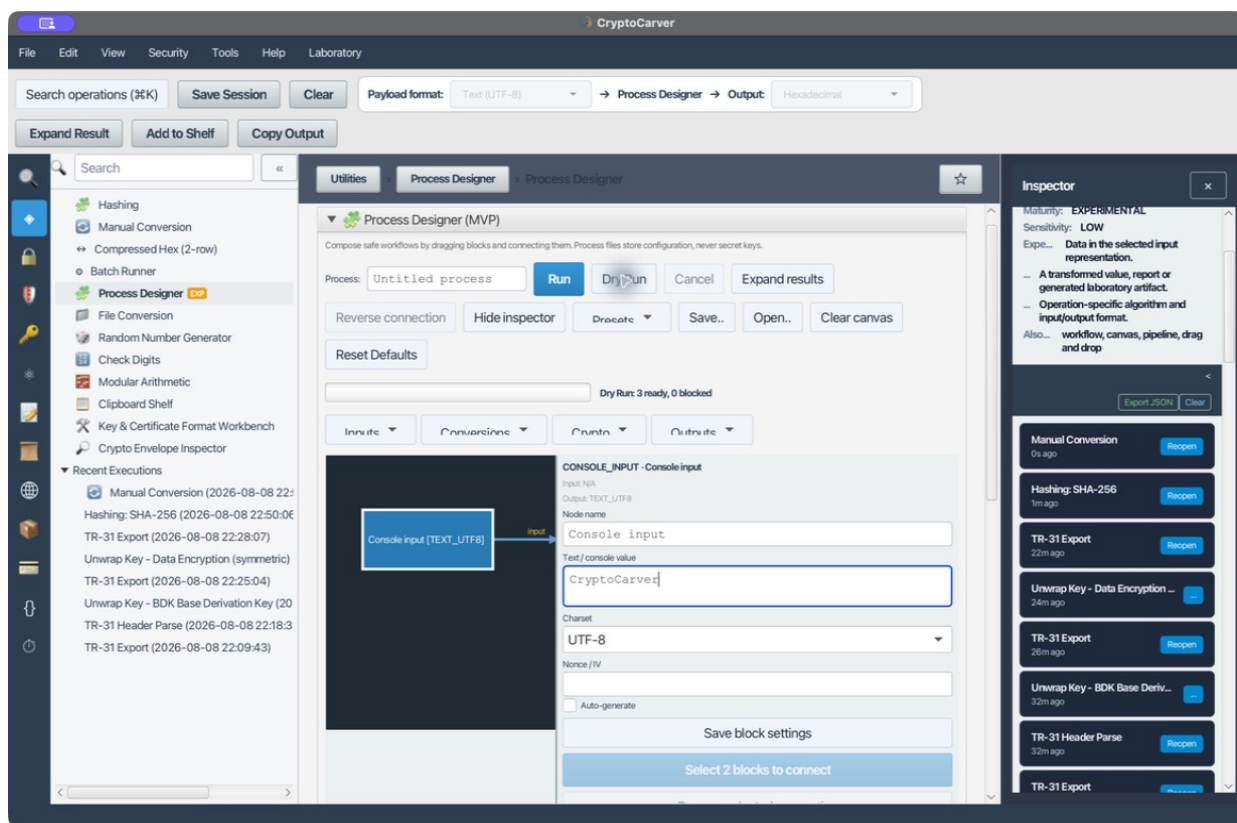
Validación	Resultado esperado
Salidas	Una por cada entrada procesada
Identificador	Conservado para reconciliar el lote
Duplicados	Mismo digest solo si los bytes son idénticos
Error de formato	Asociado a su fila, nunca convertido de forma silenciosa

Añade deliberadamente una fila malformada antes de usar un lote real: el informe debe localizarla sin atribuir otro digest a esa fila.

Caso 5: Quiero repetir una secuencia y entender cada transformación

Process Designer sirve para ver el contrato entre nodos. La captura muestra el flujo inicial Console input → SHA-256 → Console output, validado con Dry Run para la entrada CryptoCarver.

- 1 Abre Generic > Process Designer.
- 2 Comprueba que cada enlace conecta formatos compatibles; por ejemplo, texto UTF-8 con un nodo que espere bytes.
- 3 Escribe una entrada de laboratorio y ejecuta Dry Run. Revisa nodos listos, bloqueados y el número de acciones reales: debe ser cero.
- 4 Cuando el flujo básico esté estable, añade explícitamente una salida Base64 si quieres Texto → SHA-256 → Base64; no supongas que una vista hexadecimal es una codificación Base64.



Process Designer con Dry Run del flujo de hash

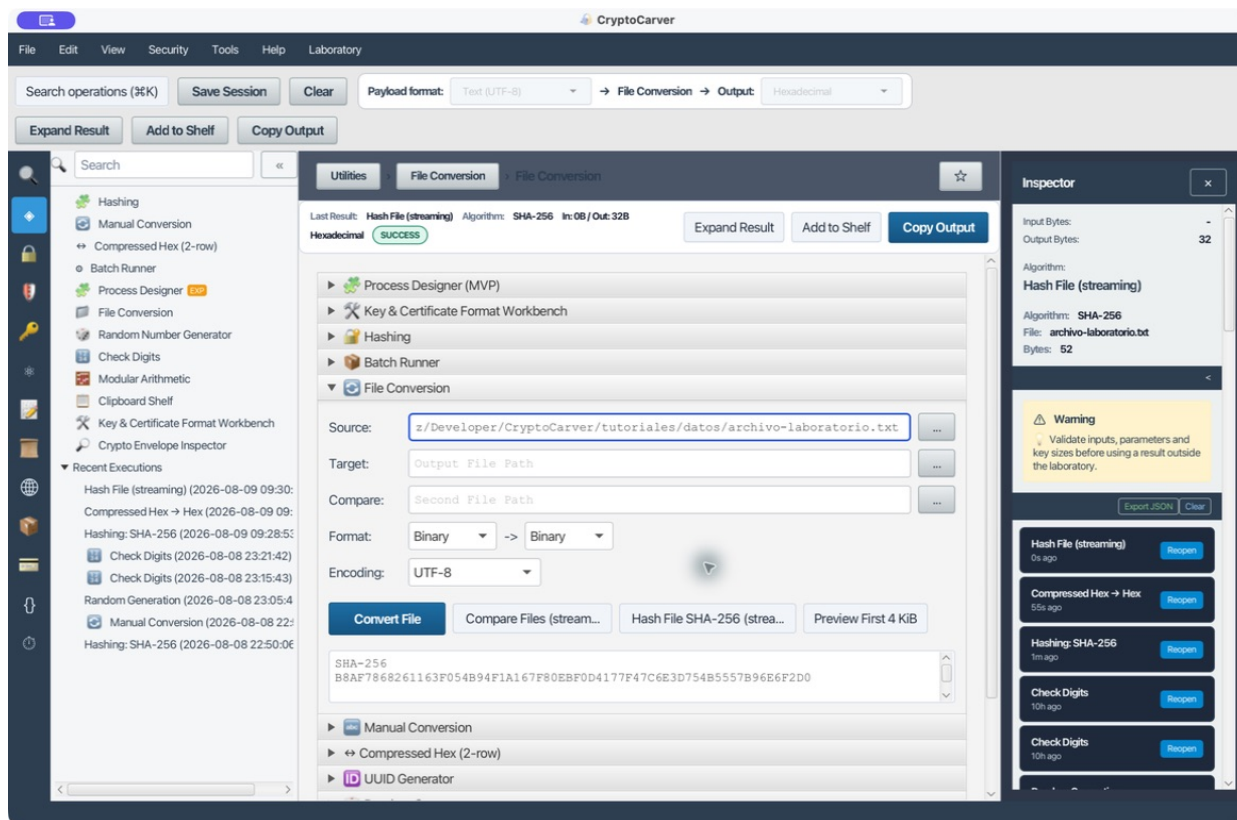
Un Dry Run valida estructura y parámetros, no sustituye un vector de prueba. Guarda al menos una pareja entrada/salida y la versión de cada nodo antes de automatizar.

Caso 6: Quiero calcular el hash de un archivo sin cargarlo entero

File Conversion permite convertir, comparar, previsualizar y calcular hashes en streaming. Para una evidencia repetible se incluye archivo-laboratorio.txt, de 52 bytes.

Parámetro	Valor
Source	tutoriales/datos/archivo-laboratorio.txt
Acción	Hash File SHA-256 (streaming)
Tamaño	52 bytes
SHA-256 esperado	B8AF7868261163F054B94F1A167F80EBF0D4177F47C6E3D754B5557B96E6F2D0

- 1 Abre Generic > File Conversion e indica la ruta del archivo de laboratorio como Source.
- 2 Pulsa Hash File SHA-256 (streaming).
- 3 Verifica simultáneamente digest y tamaño. Si el texto cambia, aunque solo sea el salto de línea final, cambiará el digest.

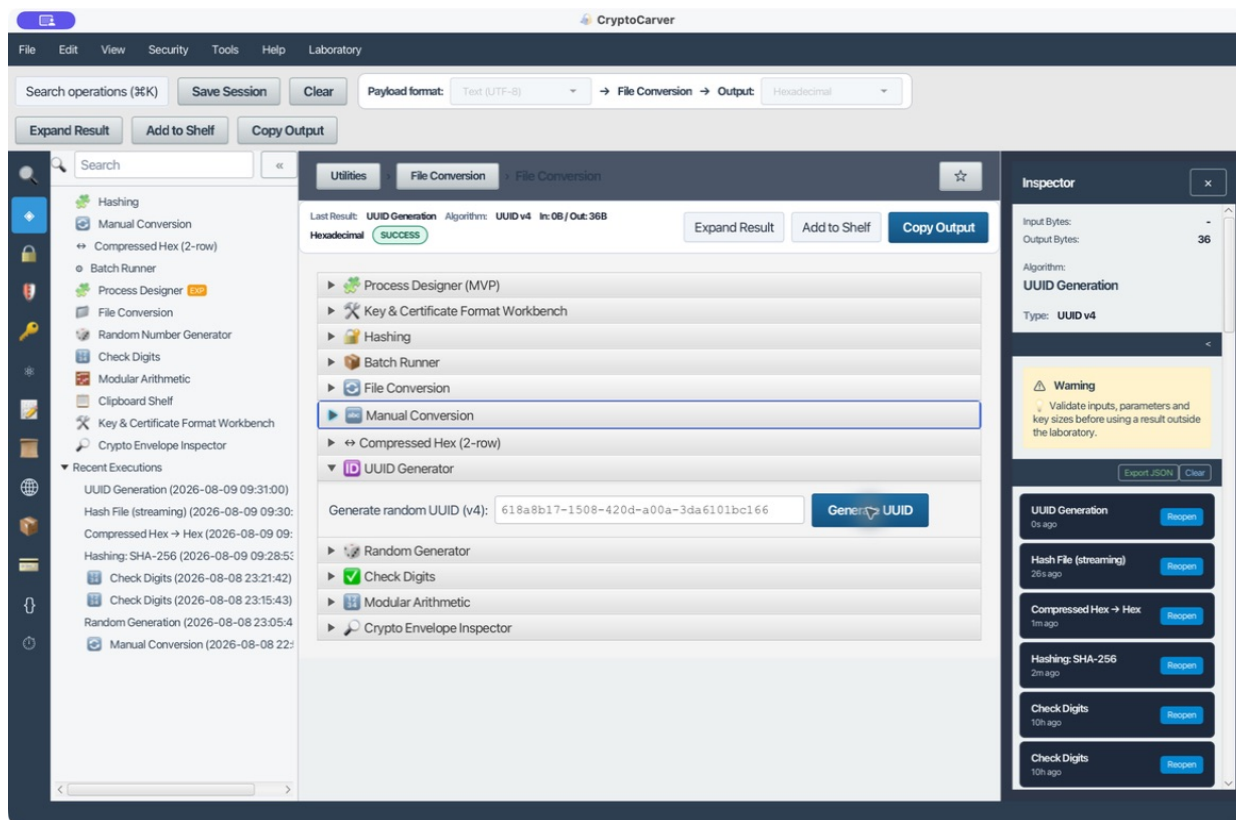


Hash streaming del archivo de laboratorio con tamaño y digest

Para convertir, establece también un destino y los formatos de origen/destino; para comparar, selecciona el segundo archivo y usa Compare Files (streaming). Nunca sobrescribas el original en la primera prueba: usa una copia y conserva el hash antes/después.

Caso 7: Quiero crear un identificador público, no una clave

En UUID Generator, Generate UUID crea un UUID v4. El valor mostrado en la captura (618a8b17-1508-420d-a00a-3da6101bc166) es una evidencia de formato, no un vector repetible: cada ejecución debe producir otro UUID.



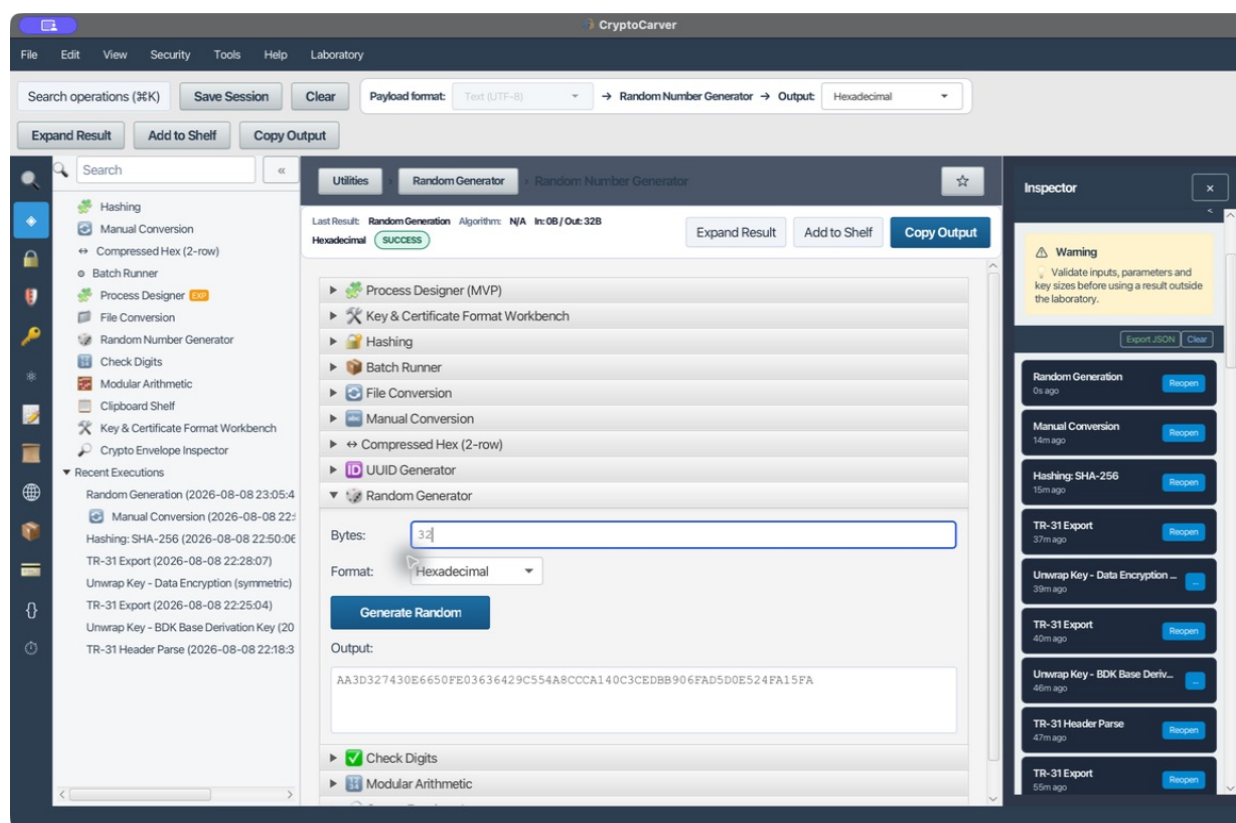
UUID v4 generado en CryptoCarver

Usa UUID para correlación, nombres de artefactos o identificadores públicos. No lo uses como clave, token de sesión de alta seguridad, nonce de AES-GCM ni sustituto de aleatoriedad criptográfica con una política concreta.

Caso 8: Quiero generar bytes aleatorios para un laboratorio

En Generic > Random Number Generator, pide una longitud de bytes, no “un número grande”. La captura genera 32 bytes en hexadecimal; su contenido es necesariamente no reproducible y solo acredita la longitud y la interfaz.

Quiero generar	Longitud habitual	Regla de seguridad
Clave AES-128	16 bytes	Secreta e impredecible
Clave AES-256	32 bytes	Secreta e impredecible
Nonce AES-GCM	12 bytes	Único por clave; no necesita ser secreto
Sal de KDF	16 bytes o más	Aleatoria y almacenada con el derivado



Generación de 32 bytes aleatorios en hexadecimal

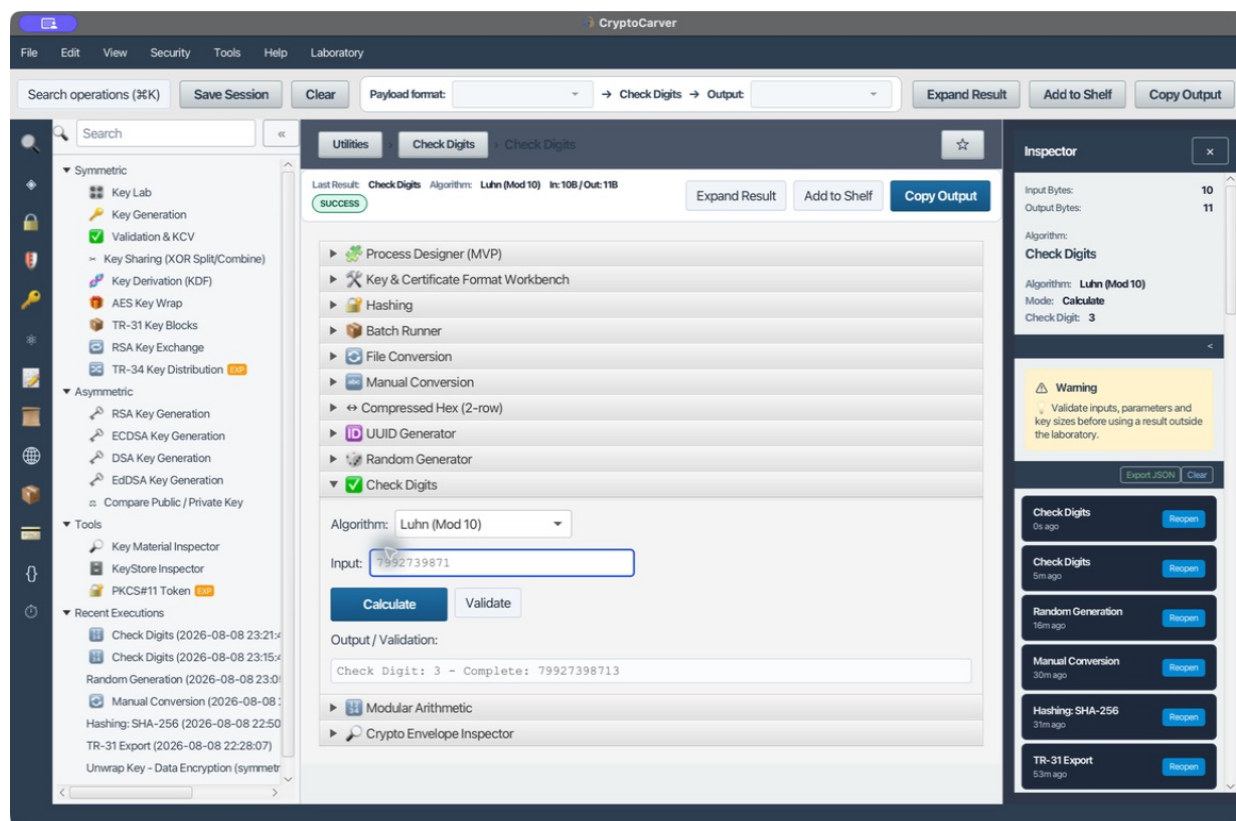
Etiqueta cada valor por finalidad y entorno. Una clave no debe reutilizarse como nonce, IV, sal o identificador. Para una receta reproducible documenta longitud, algoritmo de destino y política de almacenamiento, nunca el secreto.

Caso 9: Quiero validar una captura, no proteger datos

En Generic > Check Digits, usa Luhn para la base 7992739871.

Campo	Resultado
Base	7992739871
Dígito de control	3
Número completo	79927398713

- 1 Selecciona Luhn (Mod 10).
- 2 Introduce 7992739871 y pulsa Calculate.
- 3 La salida debe leerse Check Digit: 3 - Complete: 79927398713.
- 4 Usa Validate con el número completo; cambiar un dígito debe invalidarlo.



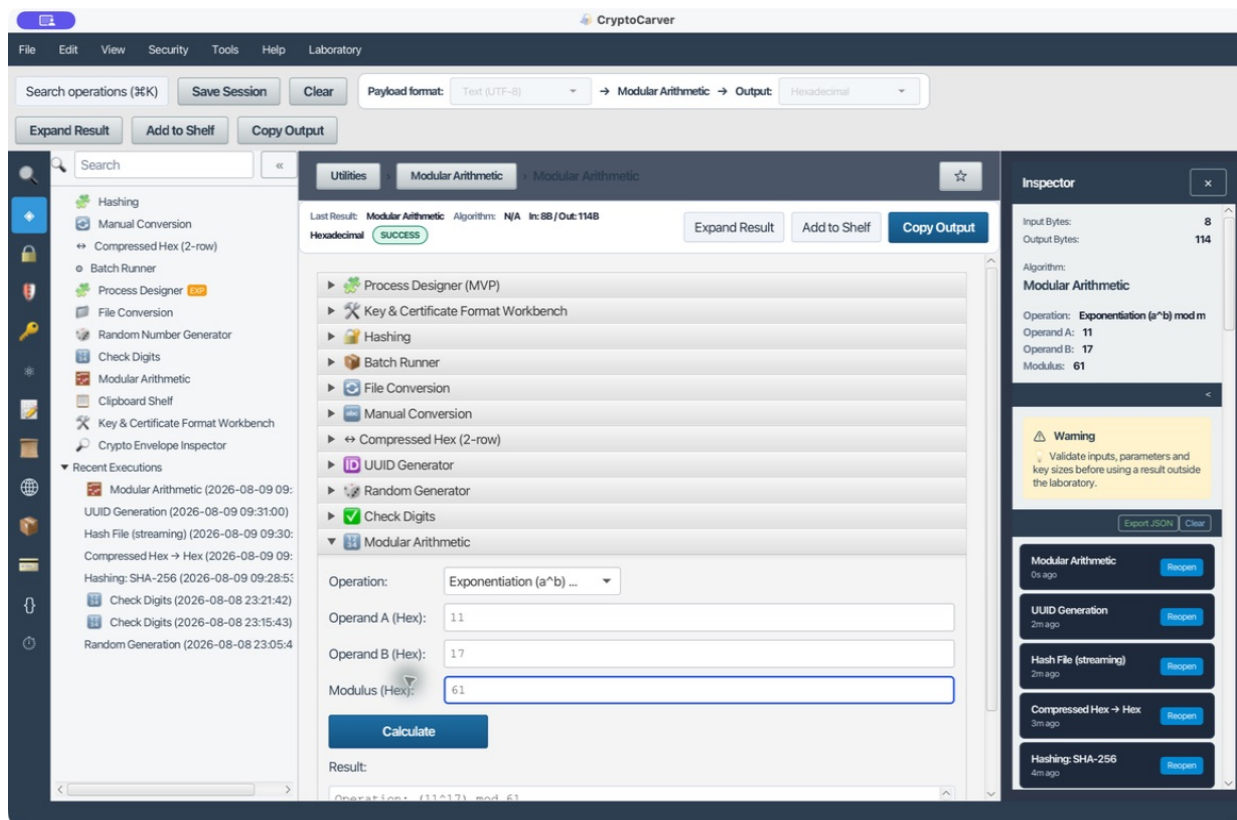
Cálculo Luhn de 7992739871 y resultado 79927398713

Luhn detecta errores de captura y algunas transposiciones. No aporta autenticidad ni confidencialidad: para resistir manipulaciones usa HMAC, CMAC o una firma digital.

Caso 10: Quiero comprobar un cálculo de protocolo

Los campos de Modular Arithmetic se introducen en hexadecimal. Para verificar el ejemplo decimal $17^{23} \bmod 97 = 7$, escribe 11, 17 y 61 respectivamente.

Campo	Valor hexadecimal	Valor decimal
Operación	Exponentiation (a^b) mod m	—
A	11	17
B	17	23
Módulo	61	97
Resultado	7	7

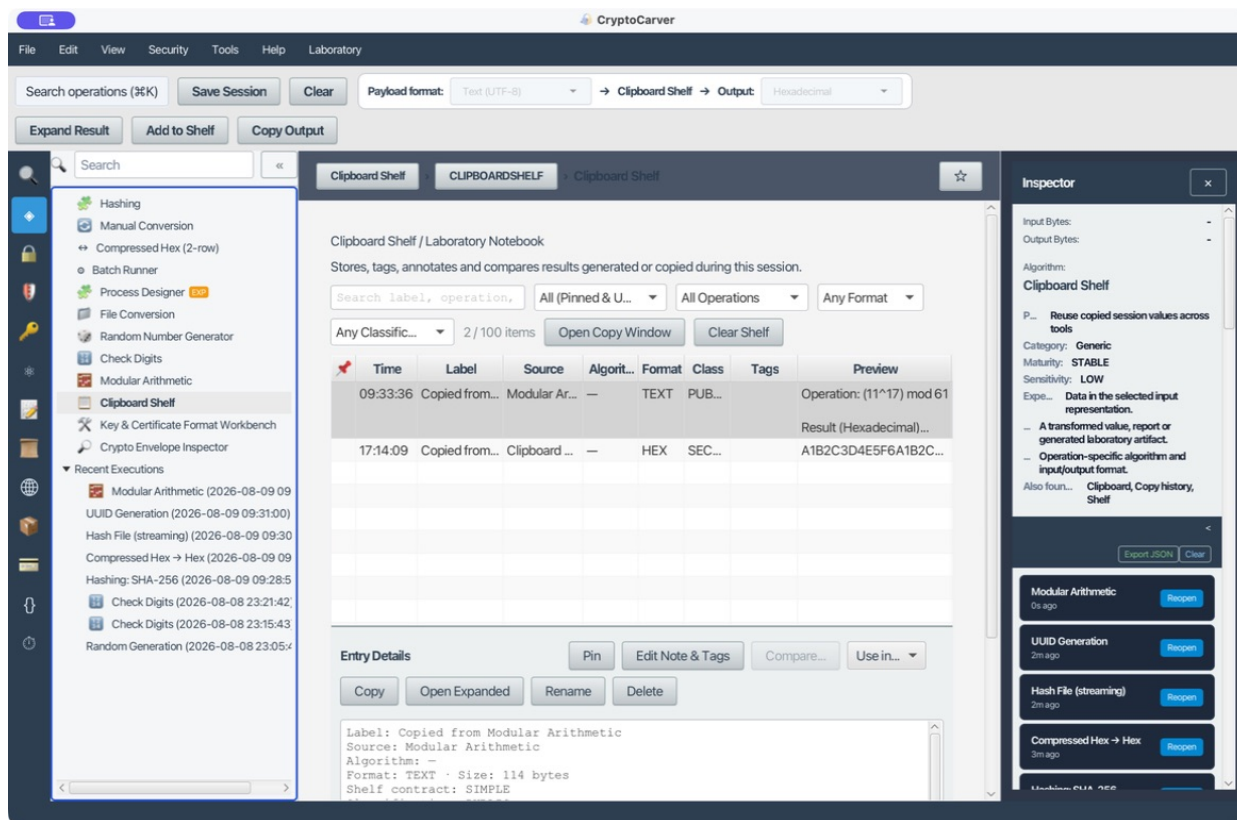


Exponenciación modular ejecutada con resultado 7

Conserva operación, base numérica, operandos y módulo. Un número decimal interpretado como hexadecimal entrega un resultado matemáticamente válido, pero para un protocolo equivocado. El módulo debe ser positivo y nunca cero.

Caso 11: Quiero reutilizar una evidencia sin perder su contexto

Clipboard Shelf funciona como cuaderno de laboratorio de la sesión. En la captura se añadió el resultado público de la aritmética modular: aparece con origen, formato TEXT, clasificación PUBLIC, tamaño y la vista previa del cálculo.



Clipboard Shelf con el resultado modular clasificado y contextualizado

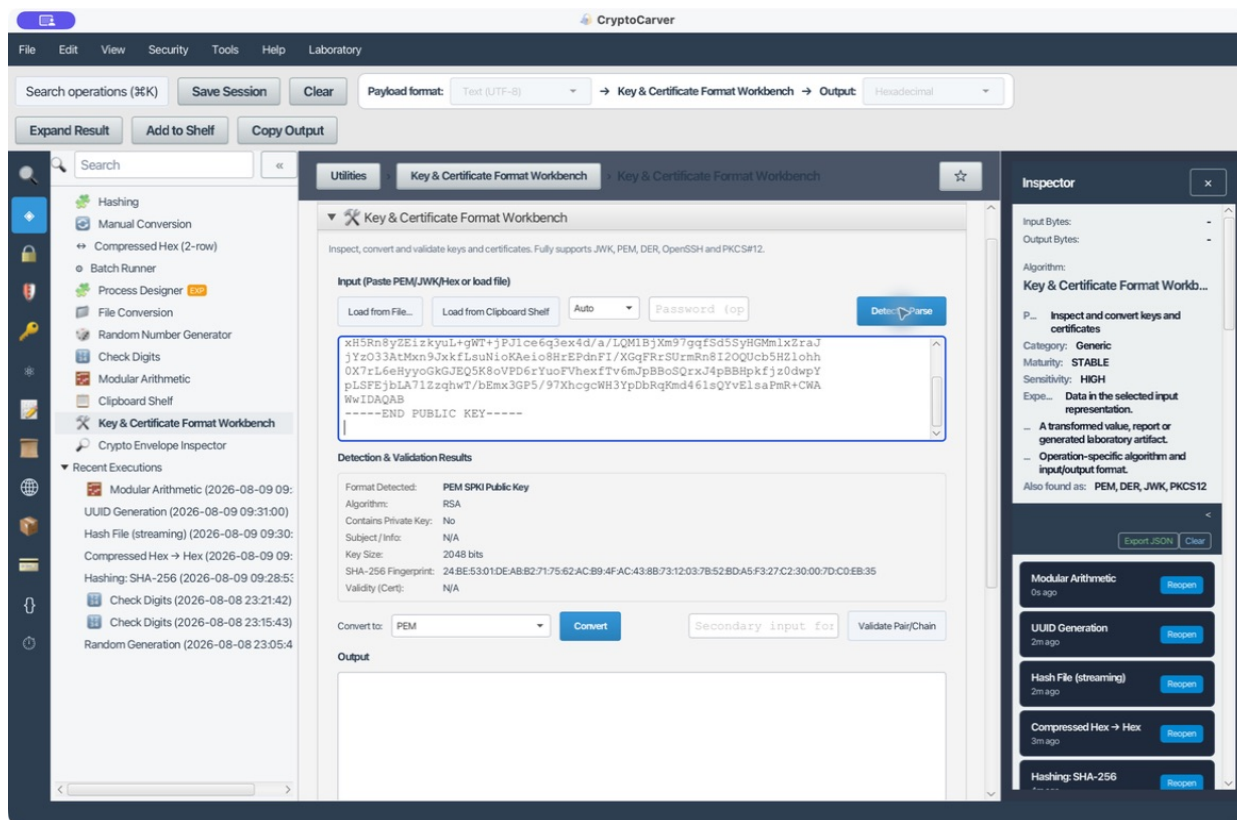
- 1 Ejecuta una operación de salida pública y pulsa Add the current result to Clipboard Shelf.
- 2 Abre Generic > Clipboard Shelf.
- 3 Verifica Source, Format, Class, tamaño y vista previa antes de reutilizar el elemento con Use selected result in an operation.
- 4 Añade etiquetas y notas que describan propósito y entorno; no copies secretos a una entrada pública.

El Shelf puede contener datos de sesiones anteriores. Revisa la clasificación antes de exportar, compartir, comparar o pegar un valor en otro módulo.

Caso 12: Quiero saber qué contiene un PEM antes de convertirlo

El Workbench entiende PEM, DER, JWK, OpenSSH y PKCS#12. Para este tutorial se incluye una clave pública PEM de laboratorio; no contiene parte privada.

- 1 Abre Generic > Key & Certificate Format Workbench.
- 2 Pega el contenido del PEM y pulsa Detect & Parse.
- 3 Comprueba los resultados de la detección: PEM SPKI Public Key, RSA, 2048 bits, sin clave privada, y el fingerprint SHA-256.
- 4 Si conviertes a DER o JWK, vuelve a detectar el resultado y verifica que representa la misma clave pública antes de usarlo en otra operación.



Workbench analizando una clave pública RSA PEM de 2048 bits

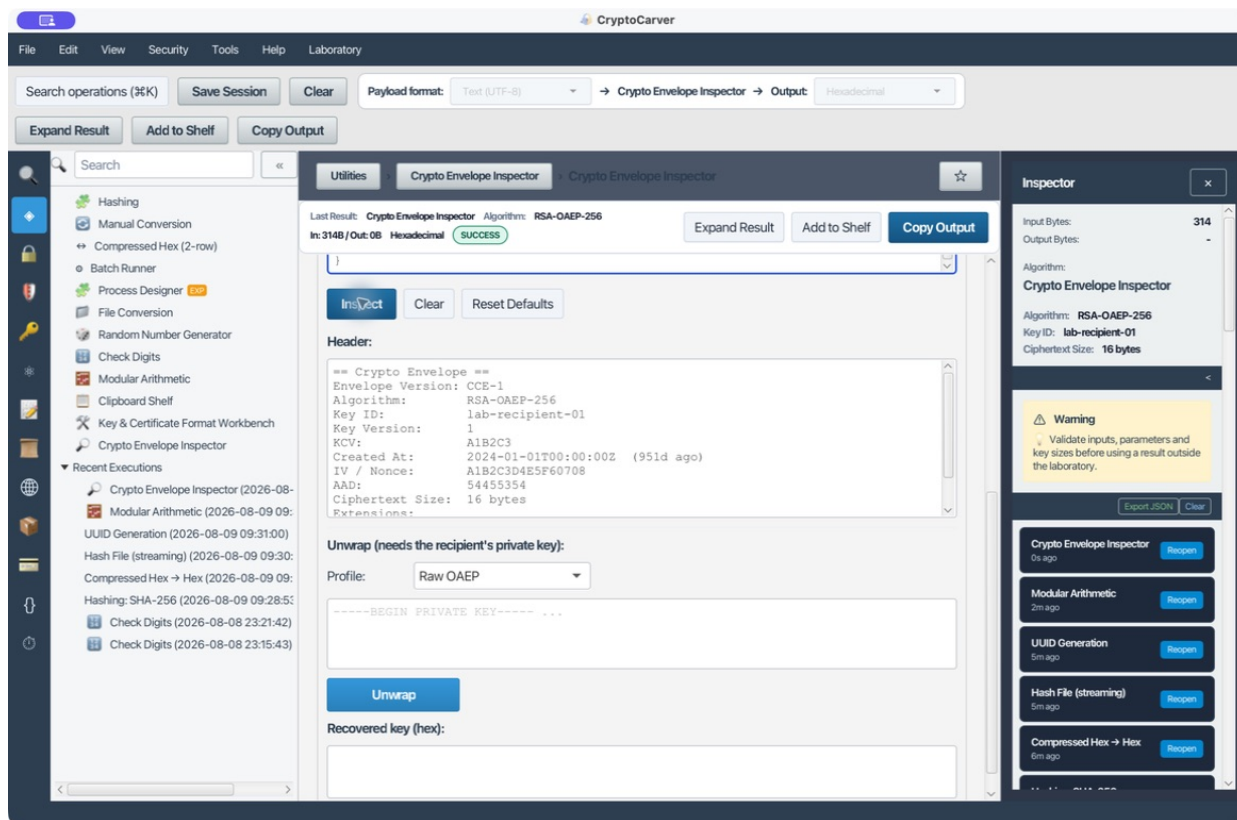
-----BEGIN PUBLIC KEY----- suele contener SubjectPublicKeyInfo; -----BEGIN PRIVATE KEY----- suele ser PKCS#8. La cabecera orienta, pero el análisis estructural determina el tipo real. Nunca conviertas una clave privada a JWK o texto plano sin confirmar el destino y la política de exposición.

Caso 13: Quiero inspeccionar un sobre criptográfico antes de intentar abrirlo

El Crypto Envelope Inspector no consume un PEM directamente: su contrato es un sobre CryptoCarver CCE-1 en JSON o forma compacta CCE1..... Por ello el PEM se usa en el Workbench del caso anterior y aquí se usa un CCE-1 válido, reproducible y sin secreto.

```
{
  "envVersion": "CCE-1",
  "alg": "RSA-OAEP-256",
  "kid": "lab-recipient-01",
  "keyVersion": 1,
  "kcv": "A1B2C3",
  "createdAt": "2024-01-01T00:00:00Z",
  "ivNonceHex": "A1B2C3D4E5F60708",
  "aadHex": "54455354",
  "ciphertextB64": "AQIDBAUGBwgJCgsMDQ4PEA==",
  "extensions": {
    "purpose": "generic-tutorial"
  }
}
```

- 1 Abre Generic > Crypto Envelope Inspector y pega el JSON.
- 2 Pulsa Inspect, sin aportar aún una clave privada.
- 3 Verifica versión, algoritmo RSA-OAEP-256, kid, versión de clave, KCV, nonce, AAD y tamaño de ciphertext (16 bytes).
- 4 Solo después de validar receptor, perfil y algoritmo debe considerarse Unwrap con la clave privada correcta. El sobre de este caso es estructural y no pretende poder desenvolverse.



Inspector de Crypto Envelope con un CCE-1 válido y metadatos completos

Inspeccionar primero permite rechazar algoritmos, destinatarios o versiones inesperadas antes de operar con una clave. No confundas esta lectura de metadatos con la validación criptográfica de que el ciphertext procede de un emisor autorizado.

Caso 14: Quiero comprobar un timestamp o dar formato a un JSON

Epoch Converter y JSON Formatter son utilidades rápidas de ventana emergente: no participan en una operación criptográfica, pero ayudan a preparar o revisar entradas (por ejemplo, un exp/iat de un JWT del tutorial de JOSE, o el cuerpo de una petición antes de firmarla).

- 1 Abre Generic > Epoch Converter. Se abre ya con el instante actual (segundos desde epoch) convertido a UTC ISO-8601.
- 2 Escribe 1700000000 en Unix Timestamp (seconds) y pulsa Convert. El resultado debe ser 2023-11-14T22:13:20Z.
- 3 Abre Generic > JSON Formatter y pega:

```
`json {"kid":"lab-01","alg":"HS256","exp":1700000000}`
```

- 1 Pulsa Format. La salida debe ser el mismo objeto indentado con dos espacios y las claves en el mismo orden.

Prueba negativa

- En Epoch Converter, escribe no-es-un-numero y pulsa Convert: el campo de resultado debe mostrar Invalid input, no un instante inventado.
- En JSON Formatter, borra la última llave de cierre } y pulsa Format: la salida debe empezar por Invalid JSON: seguido del motivo de parseo, nunca un JSON truncado silenciosamente.

Ambas herramientas registran la operación en el historial (Epoch Converter, JSON Formatter) pero no conservan el contenido introducido como detalle reutilizable; si necesitas evidencia del valor exacto, cópialo aparte antes de cerrar la ventana.

Diagnóstico rápido

Síntoma	Causa probable	Qué comprobar
SHA-256 distinto	Byte extra, charset o formato diferente	Longitud, salto final y bytes hexadecimales
Hexadecimal rechazado	Dígitos impares o carácter no hexadecimal	Pares 00–FF, sin 0x ni espacios
Base64 no decodifica	Alfabeto, padding o variante incorrecta	Base64 frente a Base64URL
Lote incompleto	Fila malformada o filtro aplicado	Recuento e identificadores de entrada
Fingerprint cambió	No es la misma clave pública	Tipo de contenedor y algoritmo
Sobre no se abre	Receptor, perfil OAEP o clave incorrectos	kid, algoritmo, versión, KCV y perfil

Checklist antes de continuar con criptografía

- ☐ Sé qué bytes exactos entran en la siguiente operación.
- ☐ He declarado la representación y no solo el “texto”.
- ☐ Tengo una entrada y salida de referencia reproducibles, o he marcado el valor como aleatorio.
- ☐ He probado una entrada deliberadamente incorrecta.
- ☐ Los datos aleatorios tienen finalidad, longitud y etiquetas distintas.
- ☐ No he confundido codificación, compresión, checksum, hash y cifrado.
- ☐ El historial, Shelf y las exportaciones no conservan secretos.

Límites de estas herramientas

Las utilidades genéricas preparan, observan y comprueban datos; no sustituyen una política criptográfica. Un hash no es una firma, un checksum no es un MAC, Base64 no es cifrado y una conversión válida no prueba que otro sistema interprete los bytes con el mismo perfil. Cuando intervengan claves, certificados, PIN o material de pago, continúa con el tutorial específico y valida el contrato de seguridad completo.